

FULL STACK DEVELOPMENT

FASE 2 | AULA 01 -

ESTRUTURA DE PROJETO EM NODE.JS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS	14

EMANIP

O QUE VEM POR AÍ?

Nesta aula, vamos explorar a estrutura de projeto em Node.js, que é essencial para criar aplicações escaláveis e bem organizadas. Entenderemos a importância de uma boa estrutura de diretórios e arquivos para facilitar a manutenção, testes e extensibilidade do projeto.



HANDS ON

Nesta aula prática, teremos como objetivo: **compreender a estrutura de projeto**, aprendendo os conceitos fundamentais de uma boa estrutura de projeto em Node.js, e **entender como o Node.js funciona por baixo dos panos**, ver como uma boa estrutura de projeto pode facilitar a manutenção e a adição de novas funcionalidades.



SAIBA MAIS

A estrutura de projeto desempenha um papel crucial na criação de aplicações escaláveis e de fácil manutenção. Organizar adequadamente os diretórios e arquivos é fundamental para garantir a sustentabilidade e a extensibilidade do código, especialmente à medida que o projeto cresce.

Organizar um projeto envolve a definição clara de onde cada componente do aplicativo deve residir. Uma estrutura bem planejada facilita a manutenção do código, a colaboração em equipe e a implementação de novas funcionalidades.

Existem algumas abordagens de arquitetura que podem nos ajudar no momento de estruturarmos o nosso projeto, mas antes de focarmos nelas, nós precisamos entender como o Node.js funciona e sua estrutura básica.

O Node.js é uma plataforma construída sobre o motor JavaScript V8 do Google Chrome, que permite usar JavaScript para desenvolver aplicações no lado do servidor.

Alguns conceitos fundamentais para entender como o Node.js funciona incluem:

1. **Single Threaded**

- O Node.js opera em um único thread, utilizando um modelo de eventos não bloqueante para gerenciar muitas conexões simultâneas sem a sobrecarga de threads múltiplas.

2. **Event Loop**

- Ele utiliza um loop de eventos para gerenciar operações assíncronas. O Event Loop orquestra a execução das funções de callback e lida com tarefas de I/O não bloqueantes, como operações de rede e de arquivos.

3. **Non-Blocking I/O**

- As operações de entrada e saída (I/O) no Node.js são não bloqueantes, o que significa que uma operação pode ser iniciada e a execução pode continuar sem esperar que a operação termine.

4. Libuv

- O Node.js utiliza a biblioteca Libuv para implementar o loop de eventos e a funcionalidade de I/O assíncrono. A Libuv abstrai as diferenças entre sistemas operacionais, fornecendo uma API consistente para o Node.js.

Antes de avançarmos para a estrutura de arquivos do Node.js, vamos entender alguns conceitos.

A **Call Stack** é uma estrutura de dados que rastreia as funções que estão sendo executadas em um determinado momento. Quando uma função é chamada, ela é adicionada ao topo da pilha. Quando essa função termina, ela é removida do topo. Esse processo é fundamental para entender como o JavaScript gerencia a execução das funções de forma síncrona.

Multi-threading refere-se à capacidade de um processador de executar múltiplas threads de execução simultaneamente. Em linguagens de programação tradicionais, cada thread pode executar uma tarefa separada, o que pode ser útil para realizar várias operações ao mesmo tempo. No entanto, em JavaScript, especialmente no Node.js, o modelo de execução é baseado em um único thread com um loop de eventos assíncrono, o que permite uma alta eficiência em operações de I/O.

A **Task Queue** é uma fila onde são armazenadas as tarefas que precisam ser executadas após a conclusão das operações atualmente na Call Stack. Quando a Call Stack está vazia, o Event Loop move as tarefas da Task Queue para a Call Stack, permitindo a execução destas. Isso é crucial para a execução assíncrona em JavaScript.

Macro tasks são grandes blocos de código que podem incluir operações como `setTimeout`, `setInterval`, e eventos I/O. Essas tarefas são colocadas na Task Queue e executadas pelo Event Loop após a conclusão das operações atualmente na Call Stack. As macro tasks são fundamentais para a gestão de operações assíncronas de maior duração.

Micro tasks são pequenas tarefas que têm prioridade mais alta do que macro tasks. Exemplos de micro tasks incluem Promise callbacks e operações realizadas

pelo MutationObserver. As micro tasks são processadas logo após a conclusão do código, atualmente em execução na Call Stack e antes das macro tasks. Isso permite um gerenciamento eficiente de operações assíncronas menores e mais rápidas.

Com esses conceitos em mente, poderemos explorar a estrutura de arquivos do Node.js e entender como esses elementos se integram para criar aplicações eficientes e escaláveis.

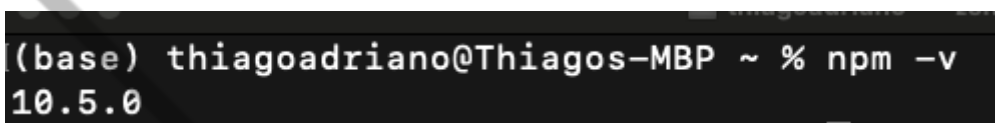
Para que possamos conhecer esta estrutura de arquivos, eu gostaria de te fazer a seguinte pergunta: o que é o NPM?

O Node Package Manager é um gerenciador de pacotes para o ambiente Node.js. Ele é uma das partes essenciais desse ecossistema e desempenha um papel fundamental no desenvolvimento de aplicativos Node.js.

Ele é uma ferramenta que permite às pessoas desenvolvedoras de Node.js compartilhar, instalar e gerenciar bibliotecas de código (pacotes) de forma eficiente e também auxilia na gestão de dependências do projeto, simplificando a inclusão de bibliotecas de terceiros em seu aplicativo.

Instalação do NPM

Quando você instala o Node.js, o NPM também é instalado automaticamente. Para verificar a versão que você tem no seu computador, basta abrir um terminal e executar o seguinte comando nele: `npm -v`



```
(base) thiagoadriano@Thiagos-MBP ~ % npm -v
10.5.0
```

Figura 1 - Verificando a versão do NPM
Fonte: elaborado pelo autor (2024)

Vejamos a seguir os principais recursos do NPM.

O NPM facilita a instalação de pacotes, permitindo que você adicione bibliotecas de código de terceiros ao seu projeto. Por exemplo, para instalar o pacote "express," você pode executar o comando `npm install express`.

Ele rastreia automaticamente as dependências do seu projeto, garantindo que as versões corretas das bibliotecas sejam instaladas. Isso ajuda a evitar conflitos de versões e problemas de compatibilidade.

Essa ferramenta permite que você defina scripts personalizados em seu arquivo `package.json` para automatizar tarefas comuns, como execução de testes, compilação ou implantação. Você pode executar esses scripts com o comando `npm run`.

NPM também permite que os(as) desenvolvedores(as) publiquem seus próprios pacotes no registro público do NPM, tornando-os disponíveis para outros(as) profissionais de desenvolvimento em todo o mundo.

Você pode especificar as versões desejadas das bibliotecas que seu projeto utiliza, permitindo um controle preciso sobre as atualizações e evitando problemas de compatibilidade.

Além de todos os recursos mencionados, ele também mantém um cache local de pacotes, o que significa que você não precisa baixar pacotes repetidamente, economizando largura de banda e tempo.

A seguir temos uma lista com os comandos básicos do NPM:

- `npm install <package>`: instala um pacote.
- `npm install`: instala todas as dependências listadas no arquivo `package.json`.
- `npm uninstall <package>`: remove um pacote.
- `npm list`: exibe uma lista das dependências instaladas.
- `npm outdated`: mostra pacotes desatualizados.
- `npm update <package>`: atualiza um pacote.
- `npm init`: inicia o processo de criação de um arquivo `package.json`.
- `npm run <script>`: executa um script personalizado definido no arquivo `package.json`.

O NPM desempenha um papel crítico no ecossistema Node.js, tornando o desenvolvimento de aplicativos mais eficiente e facilitando a colaboração na

comunidade de quem desenvolve. Ele é uma ferramenta poderosa que pode ser utilizada diariamente para gerenciar suas dependências e automatizar tarefas.

Package.json

O package.json é um arquivo de configuração fundamental em projetos Node.js. Ele contém informações sobre o projeto, como nome, versão, descrição, dependências e scripts personalizados. O NPM usa esse arquivo para gerenciar as dependências e automatizar tarefas.

Veremos as informações detalhadamente a seguir.

Nome (name): o nome do seu projeto. É geralmente um nome curto e descritivo, em letras minúsculas e sem espaços. Deve ser exclusivo no registro NPM se você planeja publicar o pacote.

```
"name": "nome-do-projeto"
```

Versão (version): a versão atual do seu projeto. Normalmente segue o formato "X.Y.Z", onde X, Y e Z são números inteiros que indicam a versão principal, secundária e de correção, respectivamente.

```
"version": "1.0.0"
```

Descrição (description): uma breve descrição do seu projeto.

```
"description": "Projeto FIAP node"
```

Dependências (dependencies): lista de pacotes de que seu projeto depende para funcionar. Cada dependência é listada com o nome do pacote e sua versão.

```
"dependencies": {  
  "express": "4.17.1",  
  "axios": "^0.21.4"  
}
```

DevDependencies (devDependencies): semelhante às dependências, mas esses pacotes são necessários apenas durante o desenvolvimento, como bibliotecas de teste ou ferramentas de compilação.

```
"devDependencies": {  
  "mocha": "^8.4.0",  
  "babel-cli": "^6.26.0"  
}
```

Scripts (scripts): define scripts personalizados que podem ser executados com o comando npm run. Você pode definir scripts para tarefas comuns, como iniciar o servidor, executar testes, compilar código, entre outros.

```
"scripts": {  
  "start": "node server.js",  
  "test": "mocha test/*.js",  
  "build": "babel src -d dist"  
}
```

Autor (author): o nome do(a) autor(a) do projeto.

```
"author": "Joao Marques"
```

Licença (license): a licença sob a qual o projeto é distribuído. Isso pode ser uma variedade de valores, como "MIT", "GPL-3.0", "Apache-2.0" etc.

```
"license": "MIT"
```

Repository (repository): o URL do repositório do seu projeto, geralmente hospedado em um serviço de controle de versão, como o GitHub.

```
"repository": {  
  "type": "git",  
  "url": "https://github.com/seu-usuario/seu-projeto.git"  
}
```

O arquivo `package.json` é essencial para gerenciar as dependências, documentação e automatização de tarefas em seu projeto Node.js.

Além disso, ele facilita a colaboração, pois outros(as) desenvolvedores(as) podem ver as informações essenciais do projeto e replicar o ambiente de desenvolvimento. Certifique-se de manter esse arquivo atualizado à medida que seu projeto evolui e suas dependências são adicionadas ou atualizadas.

Também temos o `package-lock`. O arquivo `package-lock.json` é uma parte fundamental do ecossistema Node.js e do gerenciamento de dependências usando o Node Package Manager (NPM). Ele desempenha um papel crítico no controle das versões das dependências de um projeto Node.js. Aqui estão mais detalhes sobre o `package-lock.json`:

1. **Versionamento preciso de dependências:** o `package-lock.json` armazena informações detalhadas sobre todas as dependências do seu projeto, incluindo os pacotes diretos e as dependências transitivas (ou seja, as dependências das dependências). Ele lista os nomes dos pacotes, as versões exatas e os hashes dos arquivos baixados para garantir que as versões das dependências sejam consistentes em todos os ambientes.

2. **Bloqueio de versões:** ele "trava" as versões específicas das dependências. Isso significa que, quando você instala as dependências em um ambiente de desenvolvimento, o NPM usará as versões exatas listadas no arquivo, evitando atualizações não intencionais e garantindo que todos os membros da equipe tenham as mesmas versões de dependências.

3. **Garantia de consistência:** o `package-lock.json` é particularmente útil quando você trabalha em equipes de desenvolvimento ou implanta seu aplicativo em diferentes ambientes. Garante que o comportamento do seu aplicativo seja consistente em todos os lugares, independentemente das atualizações futuras das dependências.

4. **Evita problemas de compatibilidade:** como o `package-lock.json` lista as versões exatas das dependências, ele ajuda a evitar problemas de compatibilidade

que podem surgir quando pacotes de terceiros lançam atualizações que quebram seu aplicativo. Desta forma, você pode ter confiança de que as versões das dependências permanecerão as mesmas, a menos que você as atualize manualmente.

5. **Geração automática:** ele é gerado automaticamente sempre que você instala ou atualiza pacotes em seu projeto. Isso significa que não é necessário criar ou atualizar manualmente o arquivo, pois o NPM cuida disso para você.

6. **Arquivo de bloqueio compartilhado:** o `package-lock.json` é frequentemente compartilhado no controle de versão (como Git) com o arquivo `package.json`. Isso permite que a equipe de desenvolvimento mantenha a consistência das versões de dependências em todos os membros da equipe.

Em resumo, o `package-lock.json` é uma parte essencial do gerenciamento de dependências em projetos Node.js. Ele fornece versionamento preciso, consistência de dependências e evita problemas de compatibilidade, tornando o desenvolvimento e a implantação mais previsíveis e confiáveis. Certifique-se de incluir e manter o `package-lock.json` no seu repositório para garantir a integridade das versões de dependências do seu projeto.

Por fim, temos a **`node_modules`**. Esta é a pasta onde o Node.js armazena todas as dependências e pacotes necessários para o funcionamento de um projeto.

Quando utilizamos o comando `npm install` ou `yarn install`, todas as bibliotecas listadas no arquivo `package.json` são baixadas e armazenadas nesta pasta. A `node_modules` pode se tornar extensa, pois contém não apenas os pacotes que instalamos diretamente, mas também todas as dependências desses pacotes, criando uma árvore de dependências. Por esse motivo é muito importante que ela seja colocado no arquivo `.gitignore` do seu projeto.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a estrutura de um projeto em Node.js e entendemos como ele funciona de forma detalhada.



REFERÊNCIAS

Arquitetura do Node.js: conheça seus componentes. **alura**. 2023. Disponível em: <https://www.alura.com.br/artigos/arquitetura-node-js-seus-componentes>. Acesso em: 19 ago. 2024.

MALTA, D. Como organizar e estruturar projetos com node.js. **Medium**. 2019. Disponível em: <https://medium.com/@stroklabs/como-organizar-e-estruturar-projetos-com-node-js-4845be004899>. Acesso em: 19 ago. 2024.

SILVA, T. Conhecendo o Node.js. **Medium**. /2017. Disponível em: <https://programadriano.medium.com/entendendo-o-node-js-973c9cdaf71f> . Acesso em: 19 ago. 2024.

PALAVRAS-CHAVE

PALAVRAS-CHAVE: Node.js, Estrutura de Projeto, Organização de Código, Modularidade, Manutenção de Código, Boas Práticas de Programação, Desenvolvimento de Software, Escalabilidade, Configuração de Projeto, Desenvolvimento Backend.

EXEMPLO